

Alap függvények

Print

`print(value, value, ..., sep, end)`

sep: az értékeket (values) elválasztó sztring

end: kiírás zárása pl. '\n', '\t', ',', ' ' stb.

`print('alma', 'körte', sep='-', end='\t')`

Számunkra a legfontosabb talán: end

Az "end" opció az print függvény egyik opcionális paramétere Python-ban. Alapértelmezett értéke a sortörés karakter ('\n'), ami azt jelenti, hogy minden print hívás után új sorba kerül a következő kimenet.

Viszont ha ennek a paraméternek értéket adunk, például szóközzel szeretnénk elválasztani az értékeket:

```
print("A páratlan számok:", end="")
for paratlanSzam in range(0, 21):
    if paratlanSzam % 2 == 1:
        print(paratlanSzam, end=" ")
```

kimenet:

A páratlan számok:1 3 5 7 9 11 13 15 17 19

Az "sep" paraméter (rövidítés a "separator" szóból) lehetővé teszi számunkra, hogy megadjunk egy karakterláncot, amely a print által kiírt különböző értékeket elválasztja egymástól.

`print("alma", "körte", sep="-", end="\t")`

`print("banán", "barack", sep="-", end="\t")`

kimenet:

alma-körte banán-barack //a szavak között '-' szeparátorral elválasztva, majd tabulátorral a végén

Formázott kiírás (erről később bővebben)

Az f-sztringben (vagy más néven "formatted string literal") kapcsos zárójelek között hivatkozunk a változókra, és matematikai kifejezéseket is használhatunk.

sugar = 3

`print(f"A kör kerülete: {2 * sugar * 3.14}, területe: {sugar ** 2 * 3.14}.")`

range()

A `range()` függvény egy iterálható objektumot hoz létre, amely egy adott számtartományt képvisel. Ez a függvény leggyakrabban `for` ciklusokkal használatos, de más iterációs műveletekhez is alkalmazható.

`range(start, stop, step)`

- ❑ **start (opcionális):** A tartomány kezdőértéke (alapértelmezett: 0).
- ❑ **stop (kötelező):** A tartomány végét jelző érték, amit nem tartalmaz.
- ❑ **step (opcionális):** A lépésköz értéke (alapértelmezett: 1).

```
for i in range(5):           # 0-tól 5-ig, de 5-et nem tartalmazza
    print(i)                 # Kimenet: 0, 1, 2, 3, 4
```

input()

Az `input()` függvény segítségével a felhasználótól tudunk adatot bekérni futás közben.

Hogyan működik az `input()`?

1. Felhasználótól adatot kér be:
 - Az adatot mindig szöveggént (`str` típusú) olvassa be.
2. Megadhatasz egy promptot:

A prompt az a szöveg, amely megjelenik a konzolon, hogy a felhasználó tudja, mit kell beírnia.

Típuskonverzió az `input()` után

Mivel az `input()` mindig szöveget ad vissza, számokkal történő műveletekhez explicit típuskonverzióra van szükség.

```
szam = int(input("Adj meg egy számot: "))
```

Fájl Megnyitása

A `open()` függvény segítségével tudunk fájlokat megnyitni. A függvény két fő paramétert vesz fel: a fájl nevét és a megnyitás módját.

- `r`: olvasási mód (alapértelmezett)
- `w`: írási mód (a meglévő fájl tartalmát törli)
- `a`: hozzáfűzési mód (a meglévő tartalom végéhez ír)
- `x`: kizárólagos létrehozás (ha a fájl már létezik, hibaüzenetet ad)

Fájl olvasása:

storing data on disk, and reading it back

```
f = open("file.txt", "w", encoding="utf8")
```

file variable for operations	name of file on disk (+path...)	opening mode □ <code>'r'</code> read □ <code>'w'</code> write	encoding of chars for <i>text</i> files:
---------------------------------	---------------------------------------	---	--

Files

Fájl megnyitása olvasási módban tehát:

```
file = open('example.txt', 'r', encoding = 'utf-8')
```

```
# Fájl tartalmának olvasása
```

```
content = file.read()
```

```
# Fájl bezárása
```

```
file.close()
```

```
print(content)
```

```
Fájl írása:
```

Fájl megnyitása írási módban

```
file = open('example.txt', 'w')
```

```
# Szöveg írása a fájlba
```

```
file.write('Hello, World!')
```

```
# Fájl bezárása
```

```
file.close()
```

Fájl Kezelése with Kulcsszóval

A `with` kulcsszóval megnyitott fájlokat automatikusan bezárja a Python, amikor a blokk végét ér. Ez megkönnyíti a fájlkezelést és biztosítja, hogy a fájlok mindig megfelelően bezáródjanak.

```
1 # Fájl olvasása with kulcsszóval
2 with open('example.txt', 'r') as file:
3     content = file.read()
4     print(content)
5
6 # Fájl írása with kulcsszóval
7 with open('example.txt', 'w') as file:
8     file.write('Hello, World!')
9
10 # Hozzáírás with kulcsszóval
11 with open('example.txt', 'a') as file:
12     file.write('\nHello again!')
```

split()

A `split()` függvény a Pythonban egy string objektum metódusa, amelyet arra használnak, hogy egy szöveget szétválasszanak egy megadott elválasztó (vagy a szóközők alapján) alapján, és a részeket egy **listába** tegyék.

Alapértelmezetten a `split()` függvény szóközők mentén bontja fel a sztringet, de megadhat más elválasztót is.

```
text = "Ez egy példa mondat."
```

```
words = text.split() # Alapértelmezett elválasztás: szóközők
```

```
print(words)
```

Ez a kód egy listát hoz létre a szavakból:

```
['Ez', 'egy', 'példa', 'mondat.']
```

Egyedi elválasztó megadása

Megadhat egyedi elválasztót is a `split()` függvénynek:

```
text = "alma,körte,banán"
```

```
fruits = text.split(",")
```

```
print(fruits) # ['alma', 'körte', 'banán']
```

példa érettségi feladatsorból:

```
forras = open("eloadasok.txt", "r", encoding="UTF-8")
eloadasok = []
for sor in forras:
    adatok = {} # Hozzunk létre egy üres szótárt minden új személy számára
    sor = sor.strip().split("\t") #split()-tel feldaraboljuk a sort szavakra
    adatok["nev"] = sor[0]
    adatok["honap"] = int(sor[1])
    adatok["nap"] = int(sor[2])
    adatok["sorszam"] = int(sor[3])
    adatok["hossz"] = int(sor[4])
    adatok["eloadas"] = sor[5]
```

```
adatok["eszkozok"] = sor[6]
```

```
eloadasok.append(adatok)
```

```
forras.close()
```

az ábrán egy beolvasást láthatunk, ahol a beolvasott adatokat szótárakban tároljuk el (erről később)

egyelőre csak az 5. sorig érdekes számunkra

if-elif-else

Elágazás

Akkor szükséges, ha a programban valamilyen döntési ponthoz érünk. Ha egy feltétel igaz, akkor az egyik ágon, ha nem, akkor a másik ágon (else) fut tovább a program.

A feltétel

- lehet egy logikai kifejezés, ami True (igaz) vagy False (hamis) eredményt ad.
- Használható az or (vagy) és az and (és) is.
- Az if után a nulla értékek hamisak, a nem-nulla értékek pedig igazak.
- Az üres lista, üres string is hamisat ad.

Az utasítások blokkját beljebb kezdjük, mindet ugyanannyira bent.

Pythonban nincsen többszörös elágazás, if és elseif alkalmazásával oldjuk meg.

if (feltétel1):

utasítások, amit végrehajt, ha a feltétel1 igaz

elif (feltétel2):

új eset és utasítások, amit végrehajt, ha a feltétel1 hamis, de a feltétel2 igaz (el is maradhat)

else:

utasítások, amit végrehajt, ha a feltétel1 hamis (el is maradhat)

```
szam = int(input('Adj meg egy számot! '))
```

```
if szam < 0:
```

```
    print('A megadott szám negatív.')
```

```
elif szam == 0:    //különb, Ha(tehát, ha az előtte lévő már lefutott akkor ez nem fog.)
```

```
    print('A megadott szám nulla.')
```

```
else:
```

```
    print('A megadott szám pozitív.')
```

```
print('>> A program vége! <<')
```

Python operators

Python Arithmetic Operators

Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Name	Example	Try it
+	Addition	x + y	Try it »
-	Subtraction	x - y	Try it »
*	Multiplication	x * y	Try it »
/	Division	x / y	Try it »
%	Modulus	x % y	Try it »
**	Exponentiation	x ** y	Try it »
//	Floor division	x // y	Try it »

Sztringek formázása

A Pythonban a f-strings (fűzőstringek) egy hatékony és olvasható módot kínálnak a sztringek formázására. A f-strings használata esetén a változók értékeit közvetlenül be lehet ágyazni a sztringekbe a kapcsos zárójelek {} segítségével. A változók értékei automatikusan behelyettesítődnek.

```
nev="Juliska"
```

```
eletkor=20
```

```
print(f"Szia, a nevem {nev} és {eletkor} éves vagyok.")
```

vagy akár megadhatok matematikai kifejezéseket is a {}-ben, Ebben az esetben az {1+1} rész egy kifejezés, és a Python kiértékeli azt, mielőtt beillesztené a sztringbe. Az eredmény: 1+1=2

```
1  nev="Juliska"
2  eletkor=20
3  print(f"Szia, a nevem {nev} és {eletkor} éves vagyok.")
4
5
6  print(f"1+1={1+1}")
```

ezeket nyugodtan ki lehet otthon próbálni

Alap algoritmusok

A programozási tételek gyakran használt algoritmusokat takarnak. Ezeket az algoritmusokat általában tömbökön hajtjuk végre. Az adatok persze jöhetnek billentyűzetről, fájlból vagy adatbázisból is.

Összegzés

feladat: adjuk össze a numbers lista elemeit

```
numbers=[2, 4, 5, 7, 8, 3, 1]
```

1.még a for ciklus előtt, egy kezdeti változóba inicializáljunk egy kezdeti értéket, természetesen 0 kezdőértékkel

```
ossz=0
```

2.következik a for ciklus, amellyel végigmegyünk a listán, és mely a ciklusmagot annyiszor futtatja le , ahány eleme van a listának

```
for i in numbers:
```

```
    ossz += i
```

3. printeljük ki az ossz változó értékét: `print(ossz)`

```
1  numbers=[2, 4, 5, 7, 8, 3, 1]
2
3  ossz = 0
4  for i in numbers:
5      ossz += i
6
7  print(ossz)
```

Megszámolás

Adott feltételek alapján a lista bizonyos elemeit megszámlolom.

feladat: számoljuk meg hány páros szám van a tömbben/listában

1. szamlalo kezdoerteket beallitjuk
2. végigiterálunk a numbers listán
3. egy HA feltétellel vizsgáljuk, hogy az adott szám kettővel való maradékos osztásával, 0 maradékot kapunk e, amennyiben igen, tehát a szám osztható kettővel, ezáltal páros, akkor a szamlalot növeljük 1-gyel
4. kiprinteljük az értékét a szamlalo-nak

```
1  numbers=[2, 4, 5, 7, 8, 3, 1]
2
3  szamlalo = 0
4  for i in numbers:
5      if i % 2 == 0:
6          szamlalo += 1
7  print(szamlalo)
```

Eldöntés

Szeretnénk tudni, hogy egy érték megtalálható-e egy tömbben.

feladat: van-e a tömbben 5-ös szám?

1.beállítunk egy boolean típusú változót alapértelmezetten False értékre

```
van = False
```

2. eltároljuk a keresett értéket(jelen esetben ez 5)

```
keresett_ertek = 5
```

3. for ciklussal végigiterálunk a listán, és mindegyik elemet összevetjük a keresett_ertek-kel az egyenlőség vizsgálatot két egyenlőségjel (==) jelzi

```
for i in numbers:
```

```
    if i == keresett_ertek:
```

```
        van = True
```

4.a végén ellenőrzöm, hogy a van érték igaz -e, ehhez a Pythonban a if állításnak nem szükséges expliciten megadni van == True-t, mivel az if az alatta lévő kifejezést értékeli ki logikai értékként. Ha a kifejezés igaz (azaz az érték nem nulla, nem üres, stb.), akkor a hozzárendelt kód fut le. Tehát, amikor azt írod:

```
1 numbers = [2, 4, 5, 7, 8, 3, 1]
2
3 van = False
4 keresett_ertek = 5
5 for i in numbers:
6     if i == keresett_ertek:
7         van = True
8 if van:
9     print("Igen, van 5-ös a listában")
```

Ez azt jelenti, hogy a kód a "van" változó logikai értékét értékeli ki. Ha "van" értéke True, akkor a kód a print utasítást végrehajtja. Ha "van" értéke False, akkor a print utasítás nem hajtódik végre. A van értékét már korábban beállítottad a ciklusban, ahol a feltétel alapján vagy True-ra, vagy False-ra állítódott be.

Ez az algoritmus szuper, ha például csak tartományra szeretnénk szűrni, pl van e 100-nál nagyobb szám a listában. Amikor egyetlen egy konkrét elemre szeretnénk rákérdezni elég ennyit írni:

```
1 numbers = [2, 4, 5, 7, 8, 3, 1]
2
3 keresett_ertek = 5
4 if keresett_ertek in numbers:
5     print("Igen, a keresett érték benne van a listában")
6 else:
7     print("Nem, a keresett-érték nincs a listában")
```

Ugyanez a sztringekre is működik.

II.fejezet (emelt szint)

sorted(key=)

A sorted függvény egy új listát hoz létre, amely a rendezett elemeket tartalmazza. Összetettebb rendezések esetén a sorted() függvénynek megadható egy „key” paraméter, melynek értékül egy függvényt lehet megadni. Ez a függvény a rendezés előtt az elemeket áttanszformálja. A függvény vesz egy értéket s egy új értéket ad vissza. A rendezés a függvény által visszaadott érték alapján történik.

```
1 >>> words = ['ccc', 'aaaa', 'd', 'bb']
2 >>> sorted(words)
3 ['aaaa', 'bb', 'ccc', 'd']
4 >>>
5 >>> sorted(words, key=len)
6 ['d', 'bb', 'ccc', 'aaaa']
```

Az eredeti words lista nem módosul.

A „key” értéke most a beépített „len” függvény. A szavakat a szavak hossza szerint szeretnénk rendezni. A rendezés során a len meghívódik a lista minden egyes elemére, s a rendezés a len által visszaadott érték szerint történik.

lambda kifejezés a sorted függvényben:

<https://blogboard.io/blog/knowledge/python-sorted-lambda/>

```
key=lambda t: (t["kezd_honap"], t["kezd_nap"])
```

Ez a rész egy ún. kulcsfüggvényt definiál a sorted() függvény számára. A sorted() függvény lehetőséget biztosít arra, hogy megadott szempontok alapján rendezze a listát. Ebben az esetben a rendezési kulcsot a lambda kifejezés hozza létre.

lambda t: Ez egy névtelen (anonim) függvényt hoz létre, ami egy paramétert (t-t) fogad el. A paraméter t a táborok listájában található egy-egy tábor jelöli.

(t["kezd_honap"], t["kezd_nap"]): Ez egy tuple (rendezhető és nem módosítható adattípus) létrehozása. A tuple két elemet tartalmaz: először a tábor kezdési hónapját (t["kezd_honap"]), majd a kezdési napját (t["kezd_nap"]).

Ez azt jelenti, hogy minden táborhoz egy rendezési kulcs kerül létrehozásra, amely a kezdési időpontot tartalmazza hónap és nap szerint. Amikor a sorted() függvény a táborokat rendezni próbálja, ezek a kulcsok határozzák meg a rendezés sorrendjét.

Először a hónap szerint rendez, majd ha a hónapok megegyeznek, a nap szerint. Ezzel biztosítva van, hogy a táborok kezdési időpontja szerinti növekvő sorrendben legyenek.

téglalapok 'a' és 'b' oldalának hossza

```
teglalapok = [[1, 9], [2, 3], [5, 7]]
```

'b' oldalhossz alapján rendezve

```
print(sorted(teglalapok, key=lambda t: t[1]))
```

lambda t jelen esetben a t paraméter[1] indexe, tehát a téglalapok listában lévő beágyazott listák egy-egy lista lesz úgymond a t helyett, tehát:

- 1.) $t[1]$ az a téglalapok listában kezdi az első listával (ez lesz a [1, 9]) először ez a t , tehát $t[1] = 9$ (ne felejtsük el, hogy az indexelés 0-val kezdődik)
- 2.) utána a beágyazott listák közül jön a következő (ez a [2, 3]), ebből $t[1] = 3$
- 3.) ezután pedig az utolsó ([5, 7]) következik, amiben $t[1] = 7$

- ⇒ tehát ezáltal lesz három visszatért értékünk: 9, 3, 7, és ez lesz ami alapján rendez a sorted() függvényünk. (alapértelmezetten növekvő sorrendbe rendez)
- ⇒ futassuk le a kódot, és nézzük meg mit kapunk: [[2, 3], [5, 7], [1, 9]]

terület nagysága alapján rendezve

```
print(sorted(teglalapok, key=lambda t: t[0] * t[1]))
```

reverse=True

Amennyiben egy listát szeretnél csökkenő sorrendben rendezni, használhatod a reverse=True paramétert a sorted() függvény vagy a sort() metódus esetén.

példa:

```
numbers = [5, 2, 8, 1, 3]
```

```
sorted_numbers = sorted(numbers, reverse=True)
```

```
print(sorted_numbers)
```

```
output>>[8, 5, 3, 2, 1]
```

sort()

A sorted függvény visszaad egy rendezett listát, anélkül hogy megváltoztatná az eredeti listát. Ha az eredeti listát is módosítani szeretnéd, akkor használhatod a sort metódust:

```
numbers = [4, 2, 7, 1, 9]
```

```
numbers.sort()
```

```
print(numbers)
```

Ez a kód módosítja az eredeti numbers listát és rendezetté teszi azt.

Függvények

Függvények

- Egy adott feladat elvégzését tartalmazó kódrészlet
- Csak 1x kell megírni a kódot és több helyen lehet használni
- Segít rendezni és átláthatóbbá tenni a kódot
- Fontos, hogy egy jó függvény tényleg csak 1 dolgot csinál, de azt jól

Létrehozás és tulajdonságok

- A függvény létrehozása a `def` kulcsszóval kezdődik, majd ezt követi a függvény neve
- Ezután sima zárójelek között kell megadni a függvény bemeneteit. (A bemenetek száma lehet akár 0 is)
- A bezáró zárójelet követi a szokásos kettőspont (`:`) és a behúzott szövegrészlet, ami a függvényhez tartozó kódot jelöli
- A függvényben lehetőség van a `return` paranccsal kimenetet rendelni a függvényhez

def <függvény neve>(<egyik bemenet neve>, <másik bemenet neve>, ...):

<hajts végre tetszőleges műveleteket>

<számold ki a <visszatérési érték>-et a bemenetek segítségével>

return <visszatérési érték>

Fontos korábban is használt függvényeket: `input()`, `print()`, `len()`

Függvény definíciója

```
def festek_kalkulator(x, y):
```

```
    """
```

```
    Kiszámolja az adott falfelület festéséhez
```

```
    szükséges festék mennyiségét
```

```
    """
```

```
    t = x * y
```

```
    l = t * 0.13
```

```
    return l
```

Függvény hívása

```
liter = festek_kalkulator(5, 2)
```

A függvény hívása lehet egy kifejezés része is

ar = festek_kalkulator(5, 2) * 700

```
1 >>> a
2 'Batman'
3 >>> a[-1]
4 'n'
5 >>> a[-2]
6 'a'
7 >>> a[-6]
8 'B'
9 >>> a[-3:]
10 'man'
11 >>> a[: -3]
12 'Bat'
13 >>>
```

Sztringek indexelése

(listáknál is hasonlóan)

```
4 >>> a = 'Batman'
5 >>> a
6 'Batman'
7 >>> len(a)
8 6
9 >>> a[0]
10 'B'
11 >>> a[1:4]
12 'atm'
13 >>> a[0:4]
14 'Batm'
15 >>> a[0:3]
16 'Bat'
17 >>> a[3:6]
18 'man'
19 >>> a[3:]
20 'man'
21 >>> a[:3]
22 'Bat'
23 >>> a[:]
24 'Batman'
25 >>>
```

B	a	t	m	a	n
0	1	2	3	4	5

B	a	t	m	a	n
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1

Negatív indexelés (jobbról balra):

Listáknál ugyanígy, tehát, ha például a forrásfájlunk utolsó elemét szeretnénk kiíratni(amit előzőleg eltároltunk egy listába), akkor elég lesz a [-1] indexelést használni, nem szükséges a len() függvény hívása, majd annak csökkentése a 0–val való indexelés miatt.

bónusz infók, ezeket ráér később tanulmányozni:

enumerate, index

1.)enumerate:

Ez egy beépített funkció a Pythonban.

Egy számlálót ad hozzá egy iterálhatóhoz, és egy felsorolási objektum formájában adja vissza, amely később átalakítható egy tuple-listává.

Gyakran használják for ciklusokban annak érdekében, hogy egyidejűleg bejárják a szekvencia elemeit és azok indexeit.

Példa:

```
gyumolcsok = ['alma', 'banán', 'cseresznye']
```

```
for index, ertek in enumerate(gyumolcsok):
```

```
    print(f"Index: {index}, Érték: {ertek}")
```

-->output:

Index: 0, Érték: alma

Index: 1, Érték: banán

Index: 2, Érték: cseresznye

2.)index:

list.index()

Ez egy metódus, amely a Python listáihoz tartozik.

A listában található egy elem első előfordulásának indexét keresi.

Ha az elem nincs jelen, ValueError kivételt dob.

==>Végső soron az enumerate a számlálásra, míg az index egy adott elem indexének keresésére szolgál egy listában

```
honapok = ['január', 'február', 'március', 'április', 'május', 'június']  
majus_index = honapok.index('május')  
print(majus_index)
```

>> 4

map() függvény működése

Egy **függvény**, amivel map típusú objektumok elő. A függvény két paramétert vár: az első egy másik függvény neve, a második egy bejárható objektum, például lista. A map() függvény a megadott függvényt a lista minden elemére alkalmazza. Például a map(str, [1, 2, 3]) parancs az [1, 2, 3] lista minden egyes tagjára lefuttatja az str() függvényt.

Az érettség in általában stringként vannak ábrázolva a számok, és ilyenkor nekünk az 'int' függvényt kell meghívni a map első paramétereként. pl:

```
banyato = []  
fajl = open("melyseg.txt", "r")
```

```
for sor in fajl:  
    sor = map(int, sor.strip().split())  
    print(sor)  
  
    banyato.append(sor)
```

Viszont ha ennyit írunk csak, akkor a sorok printelésénél a képernyőn egészen furcsa map objektumú kódot láthatunk:

<map object at 0x0000019FE0279810>

<map object at 0x0000019FE0279A20>

<map object at 0x0000019FE0279DB0>

<map object at 0x0000019FE0279D20>

ilyesmiket..

ezután a list() függvény ezt a módosított iterátort listává alakítja. Így a banyato lista elemei egész számok lesznek.

```
        sor = list(map(int, sor.strip().split()))  
        banyato.append(sor)  
print(banyato)
```

```
[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 3, 0, 0, 0, 0, 0, 8, 10, 10, 0, 0,  
0, 0, 0, 0, 0, 0, 0, 0, 0]]
```